

INF 208: Eingebettete und Cyber-Physische Systeme

Bütünleme-Projektbericht:

Intelligentes Patientenüberwachungs- und Sturzerkennungssystem (Smart Patient Monitoring & Fall Detection System)

Bariş Berdan Güzel

Fakultät für Ingenieurwissenschaften

Abteilung für Computertechnik (Bilgisayar Mühendisligi)

Türkisch-Deutsche Universität (TDU)

25. Juni 2026

Contents

1	Einleitung und Systemkonzept	3
2	Spezifikation und formale Modellierung	3
2.1	StateCharts (Zustandsdiagramme)	3
2.2	Petri-Netze (Petri Nets)	4
3	Simulationsumgebung und Datendiskretisierung	4
3.1	A/D-Wandlung und Datendiskretisierung	4
3.2	Bildverarbeitungssimulation	5
4	Echtzeit-Betriebssystem (RTOS) und Task-Scheduling	5
4.1	Task-Spezifikation	5
4.2	Zulässigkeitsprüfung (Scheduling-Analyse)	5
4.3	Prioritätsumkehr und Priority Inheritance Protocol	6
5	Systemevaluierung und Analytische Berechnungen	6
5.1	Energieverbrauch und Dynamic Voltage Scaling (DVS)	6
5.2	Thermisches Modell	6
5.3	Pareto-Optimierung	7
6	Open-Source- und Repositoriums-Richtlinien	7
7	Fazit und Ausblick	7

1 Einleitung und Systemkonzept

Moderne Gesundheitssysteme stehen vor der Herausforderung, eine kontinuierliche, präzise und ressourceneffiziente Patientensicherheit zu gewährleisten. Dieses Projekt befasst sich mit dem Entwurf und der formalen Evaluierung eines **Intelligenten Patientenüberwachungs- und Sturzerkennungssystems** als ein Cyber-Physisches System (CPS). Das System kombiniert biomedizinische Sensorik mit fortschrittlicher rechnergestützter Bildverarbeitung, um kritische Zustände wie Herzrhythmusstörungen (Arrhythmien) oder plötzliche Stürze autonom zu erkennen und in Echtzeit zu melden.

Als cyber-physisches System integriert diese Architektur die physische Dynamik des menschlichen Körpers (Bewegung und Elektrokardiogramm-Signale) mit den diskreten Berechnungsstufen eines eingebetteten SoC (System-on-Chip). Die wesentlichen Innovationen dieses Ansatzes liegen in der ressourceneffizienten Edge-Verarbeitung, bei der ein schlankes Bildverarbeitungsmodell direkt auf dem eingebetteten Knoten ausgeführt wird, um Bandbreite und Datenschutz zu maximieren.

2 Spezifikation und formale Modellierung

Um die Zuverlässigkeit und Korrektheit des reaktiven Verhaltens sowie des Ressourcenmanagements mathematisch nachzuweisen, werden zwei komplementäre formale Modelle eingesetzt: StateCharts für die zustandsbasierte Logik und Petri-Netze für die Nebenläufigkeit.

2.1 StateCharts (Zustandsdiagramme)

Das System wird als ein hierarchischer, reaktiver Automat spezifiziert. Die Logik unterteilt sich in drei Haupt-Superzustände (Superstates):

1. **IDLE__NORMAL**: Das System befindet sich im Ruhezustand. Es findet eine periodische Vitaldatenabfrage (EKG/Puls) im Hintergrund statt. Die Kamera verbleibt in einem Low-Power-Modus (Frame-Rate reduziert).
2. **MONITORING__ACTIVE**: Wird durch den integrierten PIR-Bewegungssensor oder eine Änderung der Pixel-Intensität ein Patient im Sichtfeld detektiert, wechselt das System in diesen Zustand. Die Kamera läuft mit vollen 30 Hz, und die Bildverarbeitungs-Pipeline (Sturzerkennung) wird aktiv geschaltet.
3. **EMERGENCY__ALERT**: Wenn entweder der Vitaldatensensor einen Herzstillstand/Tachykardie oder der optische Sensor einen Sturz detektiert, wird eine unverzügliche Zustandsüberführung in die Alarmstufe ausgelöst. Hierbei wird ein hardwarenaher Interrupt (ISR) generiert.

2.2 Petri-Netze (Petri Nets)

Für das Ressourcenmanagement und die Synchronisation des gemeinsamen Speichers (Shared Memory) wird ein Petri-Netz modelliert. Die Kamera-Task und die Sensor-Task teilen sich einen kritischen Speicherbereich für Telemetriedaten, der über einen Mutex geschützt ist.

Die Plätze (P) und Transitionen (T) sind wie folgt definiert:

- P_{Bus_Free} : Signalisiert, dass der I2C/SPI-Bus verfügbar ist (Anzahl der Token = 1).
- $P_{Sensor_Data_Ready}$: Ein neuer Vitaldatenwert liegt im Register vor.
- $P_{Frame_Captured}$: Ein Videobild wurde in den Puffer geladen.
- T_{Read_Sensor} : Sensor-Task belegt den Bus, liest die Daten und gibt den Bus wieder frei.
- $T_{Process_Frame}$: Bildverarbeitungs-Task greift auf den Speicher zu.

Dieses Modell stellt sicher, dass kein Deadlock (Verklemmung) auftritt, da die Token-Verteilung streng konservativ ist und Deadlock-Zustände strukturell ausgeschlossen werden können.

3 Simulationsumgebung und Datendiskretisierung

Die Validierung des Systems erfolgt über eine Simulationsumgebung, die in Python mittels `SimPy` (für ereignisdiskrete Abläufe) und `OpenCV` (für die Bildverarbeitungs-Pipeline) implementiert wurde.

3.1 A/D-Wandlung und Datendiskretisierung

Das analoge EKG-Signal des Patienten muss diskretisiert werden. Gemäss dem **Nyquist-Shannon-Abtasttheorem** muss die Abtastfrequenz f_s mindestens doppelt so groß sein wie die maximale Signalfrequenz f_{max} :

$$f_s \geq 2 \cdot f_{max} \quad (1)$$

Für ein medizinisches EKG-Signal wird eine maximale relevante Frequenzkomponente von 125 Hz angenommen. Daraus folgt eine gewählte Abtastrate von $f_s = 250$ Hz ($\Delta t = 4$ ms).

Die Quantisierung erfolgt über einen 12-Bit-ADC. Das Signal-Rausch-Verhältnis (SNR) bezüglich des Quantisierungsrauschens berechnet sich theoretisch wie folgt:

$$SNR = 6.02 \cdot N + 1.76 \text{ dB} = 6.02 \cdot 12 + 1.76 = 74.0 \text{ dB} \quad (2)$$

3.2 Bildverarbeitungssimulation

Die Kamera erfasst Bilder mit einer Auflösung von 640×480 Pixeln bei 30 Hz. Zur Sturzerkennung wird ein mathematischer Algorithmus zur Hintergrundsubtraktion (MOG2) gekoppelt mit einer Bounding-Box-Seitenverhältnisanalyse implementiert. Ein Sturz wird registriert, wenn die Orientierung der umschreibenden Ellipse des Patienten abrupt von vertikal zu horizontal wechselt und die vertikale Schwerpunktgeschwindigkeit einen Schwellenwert θ_{fall} überschreitet.

4 Echtzeit-Betriebssystem (RTOS) und Task-Scheduling

Die Software-Architektur basiert auf den Prinzipien eines prioritätsbasierten, präemptiven Echtzeit-Betriebssystems (wie FreeRTOS).

4.1 Task-Spezifikation

Das System besteht aus drei periodischen bzw. sporadischen Tasks:

Table 1: RTOS Task-Spezifikationen

Task	Beschreibung	Periode T_i	WCET C_i	Priorität (RM)
τ_1	EKG/Puls-Erfassung	4 ms	0.5 ms	Hoch
τ_2	Bildverarbeitung (Sturz)	33.3 ms	12.0 ms	Mittel
τ_3	Telemetrie/Ausgabe	100 ms	5.0 ms	Niedrig

4.2 Zulässigkeitsprüfung (Scheduling-Analyse)

Unter Verwendung des **Rate Monotonic (RM)** Scheduling wird die Prozessorauslastung U berechnet:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} = \frac{0.5}{4} + \frac{12}{33.3} + \frac{5}{100} = 0.125 + 0.360 + 0.050 = 0.535 \quad (3)$$

Die hinreichende Schranke nach Liu und Layland für $n = 3$ Tasks beträgt:

$$U_{bound} = 3 \cdot (2^{1/3} - 1) \approx 0.779 \quad (4)$$

Da $U = 0.535 \leq 0.779$, ist das System unter RM garantiert kollisionsfrei und ohne Verletzung von Deadlines planbar.

4.3 Prioritätsumkehr und Priority Inheritance Protocol

Wenn τ_3 den Mutex für den gemeinsamen Speicher hält und von τ_2 präemptiert wird, entsteht das Problem der **Prioritätsumkehr** (Priority Inversion), wodurch die hochprioritäre Task τ_1 blockiert wird. Zur Lösung wird das **Priority Inheritance Protocol (PIP)** implementiert. Sobald τ_1 den von τ_3 gesperrten Mutex anfordert, erbt τ_3 temporär die Priorität von τ_1 , beendet die kritische Sektion schnellstmöglich und verhindert so eine Präemption durch die mittlere Task τ_2 .

5 Systemevaluierung und Analytische Berechnungen

5.1 Energieverbrauch und Dynamic Voltage Scaling (DVS)

Die dynamische Leistungsaufnahme einer CMOS-Schaltung folgt der Formel:

$$P_{dyn} = \alpha \cdot C \cdot V^2 \cdot f \quad (5)$$

Dabei ist α die Aktivitätsrate, C die Kapazität, V die Versorgungsspannung und f die Taktfrequenz. Durch den Einsatz von **Dynamic Voltage and Frequency Scaling (DVS)** senkt das System im IDLE_NORMAL-Zustand die Taktfrequenz von 1.2 GHz ($V = 1.2$ V) auf 400 MHz ($V = 0.9$ V). Das Verhältnisergebnis der Energieeinsparung beläuft sich auf:

$$\frac{P_{idle}}{P_{active}} = \frac{0.9^2 \cdot 400 \cdot 10^6}{1.2^2 \cdot 1200 \cdot 10^6} = \frac{0.81 \cdot 400}{1.44 \cdot 1200} = \frac{324}{1728} \approx 0.1875 \implies 81.25\% \text{ Einsparung} \quad (6)$$

5.2 Thermisches Modell

Die Temperaturentwicklung des SoC unter Volllast (τ_2 aktiv) wird durch ein thermisches RC-Glied erster Ordnung beschrieben:

$$T(t) = T_{amb} + P_{dyn} \cdot R_{th} \cdot \left(1 - e^{-\frac{t}{R_{th} \cdot C_{th}}}\right) \quad (7)$$

Mit einer Umgebungstemperatur $T_{amb} = 25^\circ\text{C}$, einem thermischen Widerstand $R_{th} = 15$ K/W und einer Leistungsaufnahme $P_{dyn} = 2.5$ W konvergiert die maximale Chiptemperatur asymptotisch gegen:

$$T_{max} = 25^\circ\text{C} + (2.5 \cdot 15) = 62.5^\circ\text{C} \quad (8)$$

Dies liegt weit unterhalb der kritischen Silizium-Sperrschichttemperatur von 85°C .

5.3 Pareto-Optimierung

Es existiert ein inhärenter Trade-off zwischen der Energieeffizienz (Laufzeitverlängerung im Batteriebetrieb) und der Worst-Case Execution Time (WCET) bzw. der Latenzzeit der Sturzerkennung. Ein niedrigerer Takt spart Energie, erhöht jedoch die Erkennungslatenz. Der Entwurfsraum wird evaluiert und die optimale Konfiguration auf der **Pareto-Front** ausgewählt, die eine maximale Erkennungsverzögerung von 100 ms bei minimaler Leistungsaufnahme gewährleistet.

6 Open-Source- und Repositoriums-Richtlinien

Das Projekt wird auf GitHub als Public Repository bereitgestellt. Es umfasst gemäss den Bonus-Anforderungen:

- Eine professionelle `README.md` mit Setup-Anweisungen für die Simulationsumgebung.
- Die offizielle Open-Source-Lizenzierung (`MIT License`).
- Richtlinien für Entwicklerbeiträge (`CONTRIBUTING.md`).

7 Fazit und Ausblick

In dieser Arbeit wurde ein intelligentes Patientenüberwachungssystem formal spezifiziert, simuliert und mathematisch evaluiert. Die Ergebnisse zeigen, dass durch den Einsatz von RTOS-Scheduling-Verfahren und formalen Methoden ein hohes Maß an Zuverlässigkeit gewährleistet werden kann. Zukünftige Erweiterungen könnten die Portierung des Modells auf eine physische Plattform (z. B. FreeRTOS auf einem ESP32- oder Raspberry Pi-Knoten) umfassen.

References

- [1] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems - A Cyber-Physical Systems Approach*, MIT Press, 2017.
- [2] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, Springer, 2011.
- [3] C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment”, *Journal of the ACM*, 1973.